ЯЗЫКИ

КОМПИЛИРУЕМЫЕ

1) Код в текстовых файлах.

2) Программа компилятор «перерабатывает» написанный код в машинный код (на выходе — исполняемые файлы exe и/или dll).

3) Теперь можно запустить программу.

C, C++, C#, Rust, Go, Swift, ...

ИНТЕРПРЕТИРУЕМЫЕ

1) Код в текстовых файлах.

2) Программмма интерпретатор построчно читает и сразу выполняет команды программы

(машинный код создается и исполняется "на лету", без предварительного создания исполняемых файлов).

Python, JavaScript, Ruby, PHP, Perl, ...

ОСОБЕННОСТИ PYTHON
(ОНИ ЖЕ ПРИЧИНА ПОПУЛЯРНОСТИ)

1. Простой лаконичный синтаксис
2. Низкий порог вхождения
3. Интерпретируемый
4. Огромная экосистема библиотек и фреймворков
5. Многофункциональность
6. Большое сообщество разработчиков, активное развитие и обновление
7. Высокая степень использования в реальности
8. Автоматическое управление памятью
9. Объектно-ориентированный
10. Сильная(Строгая) Динамическая типизация
11. Кроссплатформенность

СФЕРЫ ПРИМЕНЕНИЯ PYTHON
(РАСПРОСТРАНЕННЫЕ)

1. Веб-разработка

- Создание серверной части (backend) веб-приложений
- Алгоритмы рекомендации контента
- Обработка пользовательских запросов и управление сессиями
- API и микросервисы для взаимодействия между фронтендом и бэкендом
- Веб-скрейпинг и мониторинг данных
- Веб-сокеты для создания чатов в реальном времени
- Безопасность и управление доступом пользователей
- Работа с базами данных и др. серверные процессы приложений

Примеры:

- Веб-сервисы, интернет-магазины, личные кабинеты, боты, стриминговые и видео сервисы
- Аналитические веб-инструменты для бизнеса
- Серверная часть приложений работающих на устройстве пользователя (ПК, смартфоне и тд.)

СФЕРЫ ПРИМЕНЕНИЯ PYTHON
(РАСПРОСТРАНЕННЫЕ)

2. Анализ данных и научные вычисления

- Обработка больших объёмов данных (Big Data)
- Визуализация данных для поиска закономерностей и презентаций
- Математическое моделирование и симуляции
- Работа с табличными данными (анализ, сортировка, агрегирование)
- Финансовый анализ, прогнозирование и статистический анализ данных
- Построение и анализ графов (социальные сети, логистика)

Примеры:

- Анализ пользовательского поведения
- Маркетинговая аналитика (сегментация клиентов, оптимизация рекламных кампаний)
- Прогнозирование временных рядов (например, спроса или цен)
- Обработка геопространственных данных (GIS-системы)
- Разработка и тестирование научных гипотез

СФЕРЫ ПРИМЕНЕНИЯ PYTHON
(РАСПРОСТРАНЕННЫЕ)

3. Искусственный интеллект, Машинное обучение, Нейросети

ИИ — общее название для программного обеспечения решающего сложные задачи, для решения которых обычно требуются когнитивные навыки человека (требовались раньше когда-то, отсюда и терминология).

Машинное обучение — это область искусственного интеллекта, где компьютерные алгоритмы обучаются на данных и могут делать предсказания или принимать решения без явного программирования.

Алгоритм анализирует данные, находит в них закономерности и использует их для новых входных данных.

Нейросети — область машинного обучения использующая для работы структуры (принцип) имитирующих работу нейронов, вместо традиционных алгоритмов.

ИИ

- ML (Машинное обучение)
  - Традиционные алгоритмы (не нейросети)
    - ...
  - Нейросети
    - Малые нейросети
    - Глубокое обучение нейросетей
- Экспертные системы
- Логика и планирование
- Робототехника
- NLP (Обработка естественного языка)
- Эволюционные алгоритмы

СФЕРЫ ПРИМЕНЕНИЯ PYTHON
(РАСПРОСТРАНЕННЫЕ)

3. Искусственный интеллект, Машинное обучение, Нейросети

- Обучение моделей для предсказаний (регрессия, классификация)
- Обработка изображений (распознавание лиц, анализ фотографий)
- Обработка текста и речи (перевод, чат-боты, анализ тональности)
- Построение рекомендательных систем (музыка, видео, товары)
- Обучение нейронных сетей для сложных задач (игры, биомедицинские исследования)
- Автоматизация решений на основе данных (робототехника, умные устройства)

Примеры:

- Рекомендательные системы в e-commerce
- Персонализированные ленты новостей в соц. сетях
- Голосовые ассистенты
- Распознавание объектов и сцен в видео, изображениях (например, для видеонаблюдения)
- Прогнозирование медицинских диагнозов, анализ геномных данных

...

СФЕРЫ ПРИМЕНЕНИЯ PYTHON
(РАСПРОСТРАНЕННЫЕ)

4. Автоматизация задач

- Автоматизация работы с файлами (создание, перемещение, переименование)
- Сбор данных с веб-сайтов (веб-скрейпинг) или через API
- Работа с электронными таблицами (чтение/запись данных, преобразование)
- Управление системными процессами и конфигурациями
- Автоматизация отправки email или сообщений в мессенджерах
- Регулярное выполнение задач (например, через cron или task scheduler)
- Создание пользовательских скриптов для других приложений (например, Blender, AutoCAD)

Примеры:

- Сбор информации с сайтов
- Автоматическая обработка и конвертация документов (PDF, Excel, Word)
- Массовая обработка фотографий (переименование, изменение размера)
- Генерация отчетов или периодических email-уведомлений
- Подготовка данных для других процессов (объединение, фильтрация, очистка)

СФЕРЫ ПРИМЕНЕНИЯ PYTHON
(РАСПРОСТРАНЕННЫЕ)

5. Кибербезопасность, тестирование

- Разработка инструментов для тестирования безопасности (pentesting)
- Проведение анализа уязвимостей в веб-приложениях и сетях
- Написание эксплойтов и инструментов для тестирования систем
- Парсинг и анализ логов для выявления атак и аномалий
- Имитация атак (например, DDoS, MITM) для оценки устойчивости систем
- Шифрование и дешифрование данных
- Автоматизация анализа вредоносного ПО и форензики
- Тестирование производительности приложений и систем
- Автоматизация тестирования API на безопасность
- Сканирование сетей и поиска открытых портов
- Анализ логов с серверов для выявления атак
- Имитация кибератак для проверки защиты системы (Red Team)

...

СФЕРЫ ПРИМЕНЕНИЯ PYTHON
(РАСПРОСТРАНЕННЫЕ)

6. Программирование систем, администрирование и DevOps

- Автоматизация системного администрирования
- Управление файлами и каталогами на сервере, мониторинг работы серверов и служб
- Управление сетевыми настройками, учетными записями и доступом
- Создание резервных копий и восстановление данных
- Сбор и анализ логов системы для выявления ошибок или аномалий
- Автоматизация процессов развертывания приложений
- Управление конфигурациями серверов и инфраструктуры
- Мониторинг состояния систем и приложений
- Скрипты для создания, удаления или обновления виртуальных машин и контейнеров
- Управление логами и их анализ
- Создание утилит для работы с облачными сервисами (AWS, Google Cloud, Azure)
- Резервное копирование данных и управление хранилищами
- Обновление конфигураций через Ansible, Terraform и другие инструменты

СФЕРЫ ПРИМЕНЕНИЯ PYTHON
(РАСПРОСТРАНЕННЫЕ)

7. Разработка приложений и систем для Интернета вещей (IoT)

- Управление устройствами IoT и их настройка
- Сбор данных с датчиков и их обработка
- Отправка данных на сервер или в облако
- Создание протоколов связи между устройствами (MQTT, HTTP, WebSockets)
- Автоматизация умных домов и офисов
- Анализ данных с устройств в реальном времени
- Разработка интерфейсов для управления IoT-устройствами

Примеры:

- Умные термостаты, лампы, замки и камеры видеонаблюдения
- Промышленные системы мониторинга (вибрация, температура, влажность)
- Устройства для отслеживания здоровья и фитнеса (умные браслеты, весы)
- Умные сельскохозяйственные системы (полив, мониторинг почвы)
- Транспортные системы (отслеживание местоположения, умные парковки)

СФЕРЫ ПРИМЕНЕНИЯ PYTHON
(РАСПРОСТРАНЕННЫЕ)

8. Прототипирование

9. Робототехника

СФЕРЫ ПРИМЕНЕНИЯ PYTHON
(МОЖНО, НО НЕ ЕГО КОНЕК)

Не все так однозначно. Программное обеспечение — не монолиты.

10. Desktop приложения для ПК

11. Приложения для Android или iOS

12. Игры

13. AR/VR

14. ...

ОРГАНИЗАЦИОННЫЕ МОМЕНТЫ

1. Первоначальных знаний о программировании не требуется.
2. Скучно не будет — информации много.
3. ПК любой.
4. Знание английского для обучения не требуется.
5. «Знание математики» нужно только в тех сферах где оно нужно.
6. Убеждение, что «Я гуманитарий», скорее всего косяк системы образования.
7. Все дополнительные материалы и наборы данных для практики доступны для скачивания в последней лекции этого (первого) раздела.
8. Вопросы и комментарии только приветствуются. Можно поучаствовать в обсуждении и посмотреть вопросы других — будет в плюс для самого себя.
9. Вопросы и обратная связь через «Вопросы — ответы» под каждой лекцией (там есть фильтр «Все лекции» или «текущая лекция»).

ПРИНЦИПОВ ВЕРСИОНИРОВАНИЯ МНОГО — МЯГКО СКАЗАНО…

Semantic Versioning (SemVer) – наиболее популярная схема. Она использует формат MAJOR.MINOR.PATCH и четко определяет, когда следует увеличивать ту или иную часть версии:

MAJOR — при обратно несовместимых изменениях API.

MINOR — при добавлении нового функционала совместимого с предыдущими версиями.

PATCH — при исправлении багов без изменения функциональности.

Calendar Versioning (CalVer) – основан на датах релиза.

Например YYYY.MM.DD или YY.MM и подобное Ubuntu: 20.04, 22.04, NumPy 1.24.3, pip 23.0.1, язык C(год): C99, C11, C17, C23

MAJOR.MINOR.BUILD.REVISION

Пример: 6.2.9200.16384 (Windows)

BUILD – номер сборки.

REVISION – hotfix или небольшие исправления.

Custom Versioning (полный хаос 🤯)

Пример: 1.0-a, v2024-RC1, release-final-1.5

Иногда разработчики вводят уникальные схемы для конкретных нужд.

Например, некоторые игры используют Season 2, Patch 3.5.

MAJOR.BUILD

Пример: 120.0 → 121.0

Каждое обновление – увеличение MAJOR (пример: Chrome, Firefox).

И другие…

ВЕРСИИ ПО SEMVER

…
3.13.18
3.13.19
3.13.20
3.14.0
3.14.1
…
3.14.15
3.15.0
…

4.0.0

Major (основная версия программы) — Всё переделали. Совместимость программ (написанного кода) не сохраняется. Разница в синтаксисе языка: незначительна (переход безболезненный).

Minor — Добавили новые функции. Обратная совместимость сохраняется.

Разница в синтаксисе языка: добавили новые фишки или дополнили библиотеки.

Patch — Исправили ошибки, улучшили безопасность. Не влияет на совместимость.

Примеры:

Код написанный для версии 2.x.x не совместим с версией 3.x.x и наоборот.

Код для версии 3.8.x совместим с 3.13.x, но не наоборот (хотя может повезти).

ВЕРСИИ PYTHON

3.20.17

Major Minor Patch (Micro)

...
3.13.18
3.13.19
3.13.20 → 3.13.21 → 3.13.22
3.14.0
3.14.1
...
3.14.15 → 3.14.16 → 3.14.17
3.15.0
...
3.15.21 → 3.15.22 → 3.15.23
4.0.0

Major (основная версия программы) — Всё переделали. Совместимость программ (написанного кода) не сохраняется. Разница в синтаксисе языка: незначительна (переход безболезненный).

Minor — Добавили новые функции. Обратная совместимость — нужно проверить (см. изменения в «Что нового» в релизе).

Разница в синтаксисе языка: добавили новые фишки или дополнили/изменили библиотеки.

Patch — Исправили ошибки, улучшили безопасность. Не влияет на совместимость.

Примеры:

Код написанный для python 2.x.x не совместим с python 3.x.x и наоборот.

Код для python 3.8.x очень часто совместим с 3.13.x, но не наоборот (хотя может повезти).

ИТОГ ПО ВЕРСИЯМ PYTHON

3.20.17

Major Minor Patch (Micro)

Подняли major (раз в одно или два десятилетия) — синтаксис чуть-чуть изменился (не проблема). Проекты продолжают поддерживаться на прошлой мажорной версии.

Подняли minor — ничего страшного, для прошлых миноров годами выпускаются патчи безопасности и исправления багов для поддержки проектов. (Их можно рассматривать как полу-мажорные). Синтаксис — не изменился, добавили новые возможности.

Хотите перевести свой проект на новый минор? — Посмотрите «Release Notes», возможно ваш проект изменения не затронули. А если и затронули, то скорее всего API 1-2 библиотек.

Подняли Patch (Micro) — спим спокойно (провели тесты на всякий случай, если заметки к выпуску неоднозначны). Синтаксис остался прежним.

ВЫПОЛНЕНИЕ PYTHON КОДА

Интерактивный интерпретатор в терминале. Для быстрых тестов.

pyhs@pyhs: ~

```
pyhs@pyhs:~$ python3.13
Python 3.13.0 (main, Nov 13 2024, 19:07:34)
 [GCC 13.2.0] on linux
Type "help", "copyright", "credits" or "lic
ense" for more information.
>>> 255 - 256
-1
>>> 
```

Исполнение кода интерпретатором из файла (файлов). Основной режим.

Открыть test.py ~/Ра... стол Стр. 1, Столб. 17

```
1 print(255 - 256)
```

test.py

pyhs@pyhs: ~/Рабочий стол

```
pyhs@pyhs:~/Рабочий стол$ python3.13 test.py
-1
pyhs@pyhs:~/Рабочий стол$ 
```

ПРОГРАММЫ ДЛЯ НАПИСАНИЯ И РЕДАКТИРОВАНИЯ КОДА

- Редакторы кода – простые и легковесные инструменты для работы с кодом.

  (Sublime Text, Notepad++, …,

  Visual Studio Code ??? не заслуженно здесь оказался).

- IDE (Integrated Development Environment) – мощные среды разработки с авто-дополнением, отладчиком и встроенной поддержкой языков и многими другими инструментами, котрые облегчают работу с кодом и проектом.

  (PyCharm, Visual Studio, Eclipse, …).

- IDLE (Integrated Development and Learning Environment) – встроенная среда Python для написания и тестирования кода. Простая, идет в комплекте с python, но авто-дополнение и другие функции страдают, по сравнению с профессиональными IDE.

ВСТРОЕННЫЕ (БАЗОВЫЕ) ТИПЫ ДАННЫХ

«ПРОСТЫЕ»

НЕИЗМЕНЯЕМЫЕ:

```
variable = 'Some text'     # str
variable = 123             # int
variable = 123.0           # float
variable = True            # bool
variable = None            # NoneType
variable = 5 + 3j          # complex
variable = bytes()         # байты
```

СОСТАВНЫЕ

НЕИЗМЕНЯЕМЫЕ

```
variable = (12, 13, '14', 12, True)   # tuple
```

ИЗМЕНЯЕМЫЕ

```
variable = ['10', 11, 13, 14, 11, False]   # list
variable = {'name': 'Alice', 'age': 25}   # dict
variable = {10, 11, '12', 13, 14, False}  # set
```

```
variable = bytearray()      # изменяемый массив байт
```

ПРИОРИТЕТ ОПЕРАТОРОВ И ВЫРАЖЕНИЙ

(...), [выражения...], {выражения...}	Группировка или выражение в скобках
x[2], x[0:5], x.атрибут, x(args), X()	Обращение по индексу, атрибуту, вызов функции класса
**	Возведение в степень
+x, -x, ~x	Инверсия знака +-, побитовая инверсия
*, @, /, //, %	Умножение, матричное умножение, деление, целочисленное, остаток от целочисленного деления
+, -	Сложение и вычитание
<<, >>	Побитовые сдвиги
&	Побитовая операция "И"
^	Побитовая операция "Исключающее ИЛИ"
|	Побитовая операция "ИЛИ"
in, not in, is, is not, <, <=, >, >=, !=, ==	Сравнения, проверки принадлежности и идентичности
not x	Логическое "НЕ"
and	Логическое "И"
or	Логическое "ИЛИ"
x if ? else y	Тернарный оператор
=, +=, -=, *=, /=, //=, %=, **=, :=	Присваивание, моржовый оператор (:=)

ПРОЦЕСС ВЫПОЛНЕНИЯ PYTHON КОДА

1) Когда запускается Python-скрипт, интерпретатор сначала компилирует код в байт-код — промежуточное представление, которое понятно виртуальной машине Python. Байт-код — упрощенная версия исходного кода программиста, оптимизированная для исполнения.

2) Исполнение байт-кода (интерпретация)
Скомпилированный байт-код построчно исполняется Python-интерпретатором (конкретно Python VM (PVM)), где каждый "кусок" байт-кода ~~преобразуется в машинные команды для процессора~~ выполняется как соответствующая конкретной команде машинная инструкция самого интерпретатора. В цикле чтения байт-кода происходит выполнение соответствующих инструкций интерпретатора и выполнени Си функций реализующих каскад действий для инструкций байт-кода.

3) Кеширование в .pyc
Для модулей, которые импортируются, Python автоматически сохраняет скомпилированный байт-код в файлы с расширением .pyc, находящиеся в директории __pycache__ (для ускорения повторных запусков программы).

ПОЛЕЗНЫЕ ССЫЛКИ

PEP Python Enhancement Proposals (Предложения по улучшению Python):

1) Оформление кода PEP 8:

https://peps.python.org/pep-0008/

2) Оформление строк документации PEP 257:

https://peps.python.org/pep-0257/

Официальная документация Python:

https://docs.python.org/3.13/